

PROGRAMMING · BEGINNER

C Programming Fundamentals

Start with C. Learn how programs really work.



A free, practical foundation in structured programming with C.

🕒 6-8 weeks

📁 4 modules

📖 16 lessons

[Explore the course →](#)

Concepts, worked examples, assignments and a coherent course project.

WHO IT IS FOR · WHAT YOU WILL LEARN

Your next capability.

Learn syntax, control flow, functions, arrays, pointers, structures and file handling through focused exercises and a small terminal project.



Write and debug structured C programs



Use functions, arrays, pointers and structures



Work safely with files and memory



Complete a command-line capstone



Audience

- Programming beginners
- Engineering students
- Learners preparing for C++ or embedded systems



Prerequisites

- No prior programming experience required

FROM CONCEPT TO EVIDENCE



One curriculum, three connected views

This guide summarizes the same modules and lessons shown on the course overview and in the learning workspace. The next pages explain the scope and practical evidence for every module.

DETAILED MODULE CURRICULUM · 1-2

Build the foundations.

The module names, lesson sequence and evidence below match the course learning workspace.

1 Programming and C Foundations

Build a small temperature-reporting program and learn how source becomes an executable.

1.1 Set up the C toolchain

The compiler translates C source and the linker resolves referenced functions.

1.2 Variables and data types

A variable has a type, a value and a lifetime.

1.3 Operators and expressions

Operators combine values, but their operand types influence the result.

1.4 Input and output

Formatted output requires conversion specifiers that match the arguments.

You will produce: Compilable source, command transcript and input validation cases.

2 Control Flow and Functions

Turn a one-off calculation into a reusable, testable function.

2.1 Conditions

A condition selects a path based on an expression.

2.2 Loops

A loop needs an initial state, continuation condition and progress toward termination.

2.3 Functions and scope

A function has an input contract, output contract and local state.

2.4 Debugging fundamentals

Debugging is hypothesis testing.

You will produce: Function contract and boundary-case test log.

Practice is part of every module

Work through the example, record your reasoning, complete the module assignment and compare your work with the review criteria.

DETAILED MODULE CURRICULUM · 3-4

Apply, integrate and demonstrate.

The module names, lesson sequence and evidence below match the course learning workspace.

3 Arrays, Strings and Pointers

Represent a fixed set of readings without losing track of size and lifetime.

3.1 Arrays

An array stores elements contiguously.

3.2 Strings

A C string is a character sequence ending in a null character.

3.3 Pointer foundations

A pointer stores an address associated with a type.

3.4 Pointers with arrays

Pointer arithmetic is meaningful within an array object and its one-past position.

You will produce: Array/string bounds worksheet and read-only function.

4 Structures, Files and Capstone

Assemble a terminal record manager with explicit ownership and reliable file handling.

4.1 Structures and enums

A structure groups related fields into a value; an enum names a finite set of states.

4.2 File handling

Opening, reading, writing and closing a file can fail.

4.3 Memory safety review

Review every buffer length, object lifetime and allocation owner.

4.4 Terminal application capstone

The capstone joins input validation, functions, records and persistence.

You will produce: Terminal application, reproducible tests and project README.

Practice is part of every module

Work through the example, record your reasoning, complete the module assignment and compare your work with the review criteria.

PROJECTS AND WORKING METHODS

Command-line record manager

Use functions, structures and file operations to create a small terminal application and explain how its data moves through memory.



Readable C source files



Input and boundary-case tests



Memory and file-handling explanation



Tools and methods

C compiler

VS Code

Git

Tools are learning choices, not partner endorsements.

Inside module 1: a worked example

```
#include <stdio.h>
int main(void) {
    int count = 3;
    double total = 71.0;
    printf("Average: %.2f\n", total / count);
    return 0;
}
// Compile: cc -Wall -Wextra report.c -o report
```

The output is Average: 23.67. Replacing total with an integer changes the division before formatting, so the intermediate result must be inspected.

How your work is reviewed

Show the artifact, explain your decisions, test the stated assumptions and identify limitations. Module assignments build toward the course project rather than replacing it with a single example.

DELIVERY, COMPLETION AND NEXT STEPS

Know what you are signing up for.

**Delivery**

Self-paced online · 6-8 weeks

Confirm current schedule, cohort arrangements and enrollment terms with the academy.

**Completion requirements**

Complete the required coursework and module assignments, and pass the course assessment. A course completion certificate is available after those requirements are met.

What the learning workspace includes**Follow the modules**

Expand the curriculum and open the exact lesson you need.

**Apply the concepts**

Read explanations, inspect examples and complete module practice.

**Keep your evidence**

Use resources, notes, discussion and assignments to support your learning.

Learning commitments, not outcome guarantees

Completion is not a placement, employment or performance guarantee. The course project is educational evidence; any production use requires appropriate review and validation.

Start with the curriculum. Continue with the first lesson.

Review your starting point, download this guide and explore the connected course experience.

[Go to the first lesson](#) →

Client-review edition. The static learning workspace saves demonstration progress locally; production enrollment, assessment, communications and certificate issuance are not connected.